

Diagrama de flujo. Diseño de lógica sitios web

Kevin Alexander Villamizar Santamaria

Corporación Universitaria Minuto de Dios

Facultad de Ingeniería – Ingeniería de Sistemas

NRC-8464-Ingeniería Web

Monica María Gutierrez

Bogotá, 15 de junio de 2026

Introducción

El presente trabajo tiene como objetivo diseñar un diagrama de flujo que represente la lógica de funcionamiento de un sitio web para una organización. A través de este diagrama, se analizan los componentes esenciales (cliente, servidor, DNS, protocolos HTTP/HTTPS y base de datos), sus interacciones, la importancia de la seguridad y la eficiencia en la comunicación web. El desarrollo de esta actividad permite comprender cómo la correcta estructuración de estos elementos influye directamente en la experiencia del usuario, los tiempos de carga y la protección de la información sensible.

Diagrama de flujo

A continuación se presenta el diagrama de flujo que describe la secuencia completa de interacción entre los componentes web (cliente, DNS, servidor, protocolos, base de datos, seguridad y eficiencia). El diagrama fue elaborado utilizando la herramienta draw.io (diagrams.net).

Enlace al diagrama exportado:

[Diagrama de flujo lógica sitio web](#)

Definición del propósito

Diseñar la lógica de interacción entre componentes web para una organización (ej farmacias curarte) garantizando seguridad, eficiencia, y buena experiencia del usuario.

Objetivos de estudio de componentes web:

- Identificar servidores web, clientes, protocolos (HTTP/HTTPS), DNS y base de datos necesarios

- Modelar el flujo de datos desde la resolución DNS hasta el renderizado en el navegador.
- Comparar HTTP vs HTTPS en términos de seguridad, eficiencia y caso de uso apropiados.
- Documentar buenas prácticas para desarrollar aplicaciones web

Alcance:

Sitio web transaccional con registro de usuarios, catálogo de productos y pasarela de pago. El diagrama de flujo cubre la secuencia completa de una petición típica.

Identificación de componentes

- Cliente: El navegador (Edge, Chrome, Safari, etc) que solicita y renderiza recursos web. Realiza la resolución DNS y ejecuta JavaScript.
- Servidor DNS recursivo: Traduce nombre de dominio es decir la dirección de nuestro sitio web (ej. www.farmacurarte.com) en direcciones IP. Ej. Cloudflare DNS (1,1,1,1) Google DNS (8.8.8.8).
- Servidor web: Procesa peticiones HTTP/HTTPS, entrega archivos estáticos o ejecuta lógica del lado del servidor. Ej. Apache HTTP server, nginx, Microsoft IIS
- Protocolo HTTP: Conjunto de reglas para la comunicación cliente- servidor sin cifrado HTTP/1.1, HTTP/2 (sobre texto plano en teoría, pero normalmente sobre TLS).
- Protocolo HTTPS: HTTP encapsulado en TLS (cifrado) añade seguridad, autenticación e integridad. Ej HTTPS con TLS 1.3.
- Base de datos: almacena información persistente (usuarios, productos, pedidos) Ej. MySQL, MongoDB.
- Firewall/WAF: Filtra Tráfico malicioso antes de llegar al servidor web. Ej ModSecurity, AWS WAF, Cloudflare WAF.

Análisis de interacciones

Flujo de datos entre cliente y servidor de acuerdo con el diagrama:

- Usuario escribe URL : El navegador (cliente) inicia la resolución DNS.
- Resolución DNS: El cliente consulta a un servidor DNS recursivo, que retorna la IP del servidor de destino.

- Conexión: Se establece conexión TCP (puerto 80 para HTTP, 443 para HTTPS). En HTTPS se realiza un handshake TLS adicional.
- Envío de petición: El cliente envía una solicitud HTTP (ej. GET /index.html).
- Procesamiento en servidor web: El servidor interpreta la ruta, verifica permisos y, si es necesario, consulta la base de datos.
- Respuesta: El servidor genera una respuesta HTTP con código de estado (200, 404, etc.) y el cuerpo (HTML, JSON, etc.).
- Transferencia: Los datos viajan por la red. Si es HTTPS, van cifrados; si es HTTP, en texto plano.
- Renderizado: El navegador recibe la respuesta, interpreta el HTML, descarga recursos adicionales (CSS, JS, imágenes) repitiendo pasos similares, y muestra la página al usuario.
- Interacción: El usuario hace clic o envía formularios, generando nuevas peticiones (ciclo).

Evaluación de protocolos (HTTP vs HTTPS)

Característica	HTTP	HTTPS
Seguridad	Sin cifrado (texto plano) vulnerable a ataques Man in the middle	Cifrado completo mediante TLS 1.2/1.3. Autenticación del servidor mediante certificado digital
Puerto	8080	443
Handshake inicial	Solo TCP (1 RTT)	TCP + TLS (2-3 RTT iniciales, pero TLS 1.3 reduce a 1 RTT con 0-RTT)
Certificado	No requiere	Requiere certificado SSL/TLS (de pago o gratuito como Let's Encrypt)
Eficiencia	Más rápido en la primera conexión (sin overhead criptográfico).	Overhead mínimo en conexiones largas; permite HTTP/2 y HTTP/3 (más eficientes).
SEO	Penalizado (navegador flagea como no seguro)	Favorecido por los buscadores del navegador, aumenta posicionamiento
Casos de uso apropiados	Entornos de desarrollo local, redes internas aisladas, contenido público	Todos los sitios web públicos, especialmente aquellos con login,

	sin datos sensibles	formularios, pagos o cualquier dato personal.
--	---------------------	---

Importancia de la seguridad en la comunicación web

Implicaciones de la seguridad en datos y protección de información sensible:

- **Confidencialidad:** HTTPS evita que terceros (ISP, atacantes en WiFi público) lean las credenciales, números de tarjeta de crédito o mensajes privados.
- **Integridad:** Garantiza que los datos no sean modificados durante la transmisión (Ej. inyección de anuncios maliciosos por un intermediario).
- **Autenticación:** El certificado SSL/TLS demuestra que el cliente está comunicándose con el servidor genuino y no con un impostor (phishing).
- **Cumplimiento normativo:** Regulaciones como GDPR, PCI-DSS y leyes de protección de datos exigen cifrado para información personal y financiera.
- **Confianza del usuario:** Los navegadores muestran un candado verde para HTTPS; HTTP es etiquetado como no seguro, lo que reduce la tasa de conversión.

Ejemplo

Un sitio de banca en línea sin HTTPS permitiría a cualquier persona en la misma red capturar el número de cuenta y la contraseña en texto plano. Con HTTPS, el atacante solo ve tráfico cifrado inútil.

Análisis de eficiencia

Velocidad (tiempo de carga):

DNS: Una resolución lenta (más de 100 ms) afecta la primera carga. Usar DNS rápidos (Cloudflare, Google) y caché DNS local.

Handshake TLS: Agrega latencia, pero TLS 1.3 reduce los RTT. En conexiones largas el impacto es mínimo.

Compresión: Gzip o Brotli reducen el tamaño de la respuesta hasta en un 70%, mejorando la velocidad.

Caché: Cabeceras Cache-Control permiten que el navegador utilice recursos sin descargarlos de nuevo.

HTTP/2 vs HTTP/1.1: HTTP/2 (soportado solo sobre HTTPS) permite multiplexación y compresión de cabeceras, acelerando sitios con muchos recursos.

Confiabilidad:

Redundancia de servidores: Usar balanceo de carga (Nginx, HAProxy) para que si un servidor falla, otro continúe sirviendo.

Monitorización: Health checks automáticos y reinicio de servicios caídos.

DNS anycast: Distribuye la resolución DNS mundialmente, evitando caídas regionales.

Capacidad de respuesta (escalabilidad):

Base de datos optimizada: Índices, consultas eficientes, caché de resultados (Redis, Memcached).

CDN (Content Delivery Network): Distribuye contenido estático a nivel global, reduciendo la latencia para usuarios lejanos al servidor principal.

Métrica de ejemplo:

Un sitio con HTTPS + HTTP/2 + compresión Brotli + CDN puede cargar en menos de 1.5 segundos, frente a 4+ segundos con HTTP/1.1 sin optimizaciones, reduciendo la tasa de abandono en un 40%.

Documentación de diferencias

Componente / Protocolo	Diferencia clave	Impacto en el sitio web
HTTP vs HTTPS	Cifrado y autenticación	HTTPS protege datos sensibles y mejora SEO; HTTP es inseguro y desaconsejado.
Servidor web vs Cliente	Servidor provee recursos; cliente los consume y renderiza	Sin servidor no hay datos; sin cliente no hay interacción.
DNS recursivo vs Autoritativo	El recursivo busca y cachea; el autoritativo es la fuente de verdad	El recursivo acelera consultas repetidas; el autoritativo debe ser confiable.

Base de datos relacional vs NoSQL	SQL usa esquemas fijos y transacciones ACID; NoSQL es flexible y escalable horizontalmente	SQL para sistemas financieros; NoSQL para grandes volúmenes de datos no estructurados.
Firewall vs WAF	El firewall filtra por IP/puerto; el WAF analiza el tráfico HTTP (SQLi, XSS)	El firewall es básico; el WAF protege específicamente aplicaciones web.

Conclusiones

La identificación precisa de los requisitos y el entendimiento del funcionamiento de cada componente web (DNS, servidor, protocolos, base de datos, mecanismos de seguridad) constituye la base para construir aplicaciones web robustas, escalables y seguras. El diagrama de flujo presentado no es una mera representación gráfica; es una herramienta analítica que permite visualizar el recorrido completo de los datos, detectar posibles cuellos de botella, riesgos de seguridad y oportunidades de optimización. Por ejemplo, al trazar el flujo se evidencia que omitir HTTPS expone toda la comunicación, que una caché mal configurada degrada la experiencia del usuario, o que un DNS lento puede arruinar los tiempos de carga incluso si el servidor es rápido.

En el desarrollo profesional de aplicaciones web, esta visión integral alinea la tecnología con los objetivos de negocio y las necesidades reales de los usuarios, permitiendo priorizar inversiones (como certificados SSL, CDN, balanceadores de carga) de manera informada. En conclusión, dominar el diseño lógico de sitios web mediante diagramas de flujo y análisis de componentes es una competencia esencial para cualquier ingeniero o desarrollador web que aspire a crear soluciones confiables y efectivas.

Referencias bibliográficas

Larriva Calle, D. (2019). *Proyecto Pro-Ecuador: diseño y desarrollo de marca, página web y community management, para 21 MIPYMES* (pp. 13-30). Universidad del Azuay.

Roldán Martínez, D., Valderas Aranda, P. J., & Torres Bosch, V. (2018). Microservicios: un enfoque integrado (pp. 18-46). RA-MA.

Rubiales Gómez, M. (2021). Curso de desarrollo Web: HTML, CSS y JavaScript. Edición 2021 (pp. 79-108). Anaya Multimedia.

León, F., & Díaz, G. (2019). Development of a web interface for submitting jobs to SLURM. Revista UIS Ingenierías, 18(4), 95-98.

Cloudflare. (2026). What is HTTPS? Recuperado el 14 de junio de 2026, de <https://www.cloudflare.com/learning/ssl/what-is-https/>

Google Developers. (2025). Why HTTPS matters. Recuperado de <https://developers.google.com/web/fundamentals/security/encrypt-in-transit/why-https>
Diagrams.net. (2026). draw.io (diagrams.net) [Software de diagramación en línea].
Recuperado de <https://app.diagrams.net/>